

Domain 4

Lesson 1

Python v2

Use Indentation

Python uses indentation as part of its code, specifically in any block of code that runs as part of a loop or conditional statement. Indentation is a unique feature of Python, as other programming languages use brackets. Using indentation makes code more readable and may affect code logic.

Purpose

Upon completing this project, you will become more familiar with indentation.

Steps for Completion

1. Review the code, then answer the questions regarding indentation.

```
411-indentation.py > ...
1  game_state = True
2  game_lives = 1
3  while game_lives <= 3:
4      for i in range(1,11):
5          print("You have reached position", i, "in game life", game_lives)
6      if game_state == True:
7          game_lives +=1
8  print("Thank you for playing.")
```

- a. How many times will the print statement on line 8 run? Why?
 - i. _____
 - _____
 - b. How can you change line 8 to make the print statement print with every while loop iteration?
 - i. _____
 - _____
2. What is the recommended number of spaces for indentation?
 - a. _____
 3. Label the following statement as true or false.
 - a. _____ Most Python code editors will immediately provide a tooltip telling users when their indentation is inconsistent.
 4. Why is it important to use consistent indentation in Python code?
 - a. _____
 5. Why does Python 3 not allow users to mix tabs and spaces in a single block?
 - a. _____
 - _____

Project Details

Project file

N/A

Estimated completion time

10 minutes

Video reference

Domain 4

Topic: Document Code Segments

Subtopic: Use Indentation

Objectives covered

4 Code Documentation and Structure

4.1 Document code segments

4.1.1 Use indentation

Whitespace

Whitespace refers to the sequence of spaces, tabs, or line breaks between the characters in programming code. The use of whitespace in programming languages is often flexible; however, each language has its own set of best practices for utilizing it effectively. In Python, whitespace has a more significant role than indentation and syntax. It also distinguishes between different code blocks or functions that perform separate tasks. Python's style guide, [PEP 8 – Style Guide for Python Code | peps.python.org](https://peps.python.org/pep-0008/), provides rules to follow when writing Python code.

Purpose

Upon completing this project, you will better understand whitespace.

Steps for Completion

1. Review the code, then answer the questions regarding whitespace.

```
412-white_space.py > ...
1  game_state = True
2  game_lives = 1
3  while game_lives <= 3:
4      for i in range(1,11):
5          print(f "You have reached position {i} in game life {game_lives}")
6          if game_state == True:
7              game_lives +=1
8  print("Thank you for playing.")
```

- Where might you add a space on lines 1 and 2 to make them more consistent and readable?
 - _____
 - How can you fix the unterminated string error on line 5?
 - _____
- According to Python's style guide, PEP 8, how many blank lines should precede function and class definitions?
 - _____
 - How many blank lines should separate code blocks within a function that carries out different tasks?
 - _____

Project Details

Project file

N/A

Estimated completion time

5-10 minutes

Video reference

Domain 4

Topic: Document Code Segments

Subtopic: Whitespace

Objectives covered

4 Code Documentation and Structure

4.1 Document code segments

4.1.2 Whitespace

Comments

A comment is text that follows a number sign and is either on a line by itself or at the end of a line of code. Python will ignore this text during execution, allowing users to annotate code for better understanding. Comments are marked by a number sign (#) and can either be on a line by themselves or at the end of a line of code. During execution, Python ignores comments, making them helpful in adding explanations or notes to the code without affecting its functionality.

Purpose

In this project, you will practice adding comments to code.

Steps for Completion

1. Open the **413-comments.py** file from your Domain 4 Student folder.
2. Add a comment to line 4 that reads, **#Prints positions achieved during lives**
3. Add a comment to the end of line 6 that reads, **#ranges are zero-based**
4. Add a number sign before the word, print, on line 10.
5. What does adding a number sign do to line 10?
 - a. _____
6. Save the file as **413-comments-completed**
7. Label the following statement as true or false.
 - a. _____ Each line of code can have more than one inline comment.

Project Details

Project file

413-comments.py

Estimated completion time

5-10 minutes

Video reference

Domain 4

Topic: Document Code Segments

Subtopic: Comments

Objectives covered

4 Code Documentation and Structure

4.1 Document code segments

4.1.3 Comments

Documentation Strings

Documentation strings are also known as docstrings, document functions, classes, modules, and, at times, objects. Docstrings play a critical role in explaining the purpose of an application and the components used within it. Providing documentation helps one easily understand the purpose and code of a program, which makes managing enhancements and troubleshooting much more straightforward. Ideally, docstrings should follow [PEP 257 – Docstring Conventions | `peps.python.org`](https://peps.python.org/pep-0257/), Python Enhancement Proposal, which provides docstring writing conventions.

Purpose

In this project, you will practice adding a documentation string to code.

Steps for Completion

1. Open the **414-docstrings.py** file from your Domain 4 Student folder.
2. Add four blank lines to the beginning of the code.
3. On line 1, add a docstring that reads, **"""This code shows items a player can obtain on each level in a game. Notice that the rock is not attainable on level 2."""**
4. Add a print statement on line 15 that reads, **print(_doc_)** and run the code.
5. What does adding the print statement on line 15 do?
 - a. _____
6. Save the file as **414-docstrings-completed**
7. What are the two differences between comments and docstrings?
 - a. _____
 - _____

Project Details

Project file

414-docstrings.py

Estimated completion time

5-10 minutes

Video reference

Domain 4

Topic: Document Code Segments

Subtopic: Documentation Strings

Objectives covered

4 Code Documentation and Structure

4.1 Document code segments

4.1.4 Documentation strings

Use Pydoc for Documentation

Pydoc is a Python documentation generation tool that extracts and organizes information from modules, classes, and functions. Pydoc makes it easier for people to understand modules without running countless searches on the internet for the same information.

Purpose

In this project, you will practice using pydoc to generate documentation.

Steps for Completion

1. Open a command line command prompt.
2. Run a command to get documentation on the built-in datetime module.
3. What does the first line under CLASSES read?
 - a. _____
4. Use a keyboard shortcut to end the file.
5. Run a command to send the datetime module documentation to a text file.
6. Take a screenshot of the results.
7. Save the file as **415-pydoc-completed.png** to your Domain 4 Student folder.

Project Details

Project file

N/A

Estimated completion time

5-10 minutes

Video reference

Domain 4

Topic: Document Code Segments

Subtopic: Use Pydoc for Documentation

Objectives covered

4 Code Documentation and Structure

4.1 Document code segments

4.1.5 Generate documentation by using pydoc

Review 4.1

Purpose

Upon completing this project, you will better understand how to add documentation to code files created in Python.

Steps for Completion

1. Open the **415-review.py** file from your Domain 4 Student folder.
2. Add text near the top of the file explaining the purpose of the file.
3. Convert the text you have added to a documentation string.
4. Add a line of code that prints the documentation.
5. Compare your results to the answer in the next video.

Project Details

Project file

415-review.py

Estimated completion time

5 minutes

Video reference

Domain 4

Topic: Document Code Segments

Subtopic: Use Pydoc for
Documentation; Review 4.1

Objectives covered

4 Code Documentation and Structure

4.1 Document code segments

Domain 4 Lesson 2

Python v2

Call Signatures

A function is a block of code that can be called multiple times within an app and is best used when a process needs to be repeated. By using functions, developers can avoid repeating the same code repeatedly, making their programs more efficient and easier to manage.

Purpose

Upon completing this project, you will become more familiar with call signatures.

Steps for Completion

1. Review the code, then answer the questions regarding functions and call statements.

```
421-signatures.py > ...
1  def total_score(score, multiplier):
2      return score * multiplier
3
4  print(total_score(3000,1.5))
5  print(total_score(4000,2))
```

- a. On line 4, which number represents the score and which represents the multiplier?
 - i. _____
- b. What will be the output from running this code?
 - i. _____
- c. What type of statements in this file call the function?
 - i. _____

Project Details

Project file

N/A

Estimated completion time

5 minutes

Video reference

Domain 4

Topic: Function Definitions

Subtopic: Call Signatures

Objectives covered

4 Code Documentation and Structure

4.2 Construct and analyze code segments that include function definitions

4.2.1 Call signatures

Default Values

When writing code in Python, it is possible to set default values for function arguments. These values are utilized when the caller does not provide a value for them. Default values can save time and prevent errors when calling functions repeatedly with the same values.

Purpose

Upon completing this project, you will better understand default values.

Steps for Completion

1. Open the **422-default.py** file from your Domain 4 Student folder.
2. Make **1.5** the default value for the multiplier argument.
3. On line 4, remove the multiplier from the function call.
4. Save the file as **422-default-completed**
5. Review the code, then answer the question regarding the new print statement.

```
422-default.py > ...
1 def total_score(score, multiplier=1.5):
2     return score * multiplier
3
4 print(total_score(3000,))
5 print(total_score(4000,2))
6 print(total_score(multiplier=3, score=15000))
```

- a. What will be the output of running this code?

i. _____

Project Details

Project file

422-default.py

Estimated completion time

5-10 minutes

Video reference

Domain 4

Topic: Function Definitions

Subtopic: Default Values

Objectives covered

4 Code Documentation and Structure

4.2 Construct and analyze code segments that include function definitions

4.2.2 Default values

Use return

In Python, functions may either return a value or not, depending on the task. When they return a value, it can be saved in a variable and used later for further calculations or other purposes. Thus, functions save time by eliminating the need to write the same code multiple times and provide a convenient way to organize and reuse code.

Purpose

Upon completing this project, you will become more familiar with using the return keyword.

Steps for Completion

1. Review the code, then answer the questions regarding the functions.

```
423-return.py > ...
1  def total_score(score, multiplier):
2      return score * multiplier
3
4  def welcome():
5      print("Welcome to the next level")
6
7  score1 = total_score(3000,2)
8  score2 = total_score(2000,1.5)
9
10 welcome()
11 print(f"Your two scores are {score1} and {score2}.")
12
```

- a. Which function would warrant a value stored for future use, the total score or the welcome message function?

i. _____

- b. Why does this code need the return statement on line 2?

i. _____

2. What is a void function?

a. _____

Project Details

Project file

N/A

Estimated completion time

5-10 minutes

Video reference

Domain 4

Topic: Function Definitions

Subtopic: return

Objectives covered

4 Code Documentation and Structure

4.2 Construct and analyze code segments that include function definitions

4.2.3 return

Use def

When defining a function in Python, the keyword function is not used, as in many other programming languages. Defining a function increases the efficiency of code and helps prevent errors.

Purpose

In this project, you will practice defining a function using the def keyword.

Steps for Completion

1. Open the **424-def.py** file from your Domain 4 Student folder.
2. On line 2, define a function named **calculate_score** and add two arguments in parentheses: **score** and **multiplier**
3. On line 3 add a return statement that returns the **score** times the **multiplier**
4. On line 5, use a print statement to call the **calculate_score** function, making the score **3500** and the multiplier **2**
5. Run the code. What is the output of running this code?
 - a. _____
6. Copy the print statement on line 5 to line 6, changing the score to **5500** and the multiplier to **1.6**
7. Run the code. What is the output of running this code?
 - a. _____
8. Save the file as **424-def-completed**

Project Details

Project file

424-def.py

Estimated completion time

10 minutes

Video reference

Domain 4

Topic: Function Definitions

Subtopic: def

Objectives covered

4 Code Documentation and Structure

4.2 Construct and analyze code segments that include function definitions

4.2.4 def

Use pass in Functions

The pass keyword acts as a temporary placeholder when testing a function. It is useful when all the necessary information to complete a function’s calculation is unavailable, as it allows users to ensure the remaining code works as expected.

Purpose

Upon completing this project, you will become more familiar with using pass in a function.

Steps for Completion

1. Open the **425-pass.py** file from your Domain 4 Student folder.
2. Run the code.
3. Enter the amount **35000** for the first input statement.
4. Enter the amount **1500** for the second input statement.
5. What is the total?
 a. _____
6. What has the pass statement in this code block allowed you to do?
 a. _____

Project Details

Project file

425-pass.py

Estimated completion time

5 minutes

Video reference

Domain 4

Topic: Function Definitions

Subtopic: Use pass in Functions

Objectives covered

4 Code Documentation and Structure

4.2 Construct and analyze code segments that include function definitions

4.2.5 pass

Review 4.2

Purpose

Upon completing this project, you will better understand how to define and work with functions.

Steps for Completion

1. Open the **425-analyze.py** file from your Domain 4 Student folder.
2. When calling the `calculate_total` function, must a tax rate be specified?
 - a. _____
3. What will be the amounts of the two subtotals when the code runs?
 - a. _____
4. Return to the video portion of the course to compare your answers to the ones in the videos.

Project Details

Project file

425-analyze.py

Estimated completion time

5 minutes

Video reference

Domain 4

Topic: Function Definitions

Subtopic: Use pass in Functions;
Review 4.2

Objectives covered

4 Code Documentation and Structure

4.2 Construct and analyze code segments that include function definitions